

■ Recursion in Java

A Spoon-Feed Guide — From Zero to Confident

■ Beginner-Friendly

■ Code Examples

■ Step-by-Step

■ Practice Q's

■ What's Inside

1. What is Recursion?
2. The Two Golden Rules
3. How Java Executes Recursion (Call Stack)
4. Example 1 – Factorial
5. Example 2 – Fibonacci
6. Example 3 – Sum of Array
7. Example 4 – Reverse a String
8. Recursion vs Iteration
9. Common Mistakes & Tips
10. Practice Questions

1 ■ ■ What is Recursion?

Recursion is when a **method calls itself** to solve a smaller version of the same problem. Think of it like a set of Russian nesting dolls — each doll contains a smaller doll, until you reach the smallest one.

Real-World Analogy	Recursion Idea
■ A dictionary defines a word using other words	A function solves a problem by calling itself
■ Two mirrors facing each other	A function calling itself inside itself
■ Peeling an onion layer by layer	Breaking a big problem into smaller sub-problems

Simple definition: A recursive method = a method that **calls itself** with a **simpler input** until it reaches a **stopping point**.

2 ■ ■ The Two Golden Rules of Recursion

Base Case ■	The condition where recursion STOPS. Without this, your program runs forever (Stack Overflow!). It's like the smallest Russian doll — it cannot be opened further.
Recursive Case ■	The part where the method calls itself with a SMALLER or SIMPLER input, moving towards the base case. Every recursive call must get closer to the base case.

Template of every recursive method:

```
returnType methodName(parameters) { // BASE CASE – stop here if (simplest condition) { return simple answer; } // RECURSIVE CASE – call yourself with smaller input return methodName(smaller input); }
```

3 ■ ■ How Java Executes Recursion (The Call Stack)

Every time a method is called in Java, it gets pushed onto a **Call Stack**. Think of the call stack like a pile of plates — the last plate placed is the first one removed (LIFO).

For `factorial(3)`, the call stack looks like this:

Call Stack (Growing ▼)	Waiting for...
<code>factorial(3)</code>	waiting for <code>factorial(2)</code>
<code>factorial(2)</code>	waiting for <code>factorial(1)</code>
<code>factorial(1)</code>	waiting for <code>factorial(0)</code>
<code>factorial(0) ← BASE CASE</code>	returns 1 ■

■ ■ **Stack Overflow Error** — If you forget the base case or the input never reaches the base case, Java keeps stacking calls until memory runs out and throws a **StackOverflowError**.

4 ■ ■ Example 1 — Factorial of a Number

What is factorial?

$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$ In general: $n! = n \times (n-1)!$ and $0! = 1$

Step-by-step thinking:

1 Can I solve `factorial(5)` directly? — Too complex.

2 I know `factorial(5) = 5 × factorial(4)`

3 I know `factorial(4) = 4 × factorial(3)` ... and so on

4 Base case: `factorial(0) = 1` (stop here!)

Java Code:

```
public class Factorial { // Recursive method
    static int factorial(int n) { //
        BASE CASE if (n == 0) { return 1; // 0! is defined as 1 } // RECURSIVE CASE
        return n * factorial(n - 1); // e.g. 5 * factorial(4) }
    public static void
    main(String[] args) { System.out.println(factorial(5)); // Output: 120
        System.out.println(factorial(0)); // Output: 1 } }
```

■ **Trace it:** `factorial(3) → 3 × factorial(2) → 3 × 2 × factorial(1) → 3 × 2 × 1 × factorial(0) → 3 × 2 × 1 × 1 = 6`

5 ■ ■ Example 2 — Fibonacci Series

What is Fibonacci?

0, 1, 1, 2, 3, 5, 8, 13, 21 ... Each number = sum of the previous two numbers. $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ Base cases: $\text{fib}(0)=0$, $\text{fib}(1)=1$

Java Code:

```
public class Fibonacci { static int fib(int n) { // BASE CASES (two this time!)
if (n == 0) return 0; if (n == 1) return 1; // RECURSIVE CASE return fib(n - 1)
+ fib(n - 2); } public static void main(String[] args) { for (int i = 0; i < 8;
i++) { System.out.print(fib(i) + " "); } // Output: 0 1 1 2 3 5 8 13 } }
```

■ ■ **Note:** Fibonacci is simple but **inefficient** recursively — $\text{fib}(5)$ recalculates $\text{fib}(3)$ multiple times. For large numbers, use Dynamic Programming (memoization). But for learning recursion, it's perfect!

6 ■ ■ Example 3 — Sum of an Array

Find the sum of all elements in an array using recursion. Idea: $\text{sum}(\text{arr}) = \text{arr}[0] + \text{sum}(\text{rest of array})$

Java Code:

```
public class ArraySum { static int sum(int[] arr, int index) { // BASE CASE:
reached end of array if (index == arr.length) { return 0; } // RECURSIVE CASE:
current + sum of rest return arr[index] + sum(arr, index + 1); } public static
void main(String[] args) { int[] numbers = {1, 2, 3, 4, 5};
System.out.println(sum(numbers, 0)); // Output: 15 } }
```

■ **Trace:** $\text{sum}(\{1,2,3\}, 0) = 1 + \text{sum}(\{1,2,3\}, 1) = 1 + 2 + \text{sum}(\{1,2,3\}, 2) = 1 + 2 + 3 + 0 = 6$

7 ■ ■ Example 4 — Reverse a String

Reverse "hello" → "olleh". Idea: reverse(str) = last character + reverse(str without last character)

Java Code:

```
public class ReverseString { static String reverse(String str) { // BASE CASE:
empty or single character if (str.length() <= 1) { return str; } // RECURSIVE
CASE // Take last char + reverse the rest return str.charAt(str.length() - 1) +
reverse(str.substring(0, str.length() - 1)); } public static void main(String[]
args) { System.out.println(reverse("hello")); // Output: olleh
System.out.println(reverse("java")); // Output: avaj } }
```

8 ■ ■ Recursion vs Iteration

Feature	Recursion	Iteration (Loop)
Code Length	Usually shorter	Can be longer
Readability	Often cleaner	Straightforward
Performance	Slower (stack overhead)	Faster
Memory	Uses call stack	Less memory
Risk	Stack Overflow	Infinite loop
Best For	Trees, graphs, divide & conquer	Simple repetition

■ Use recursion when the problem is **naturally recursive** (trees, graphs, puzzles). Use iteration for simple counted repetition.

9 ■ Common Mistakes & Pro Tips

■ Mistake 1: Forgetting the Base Case

Without a base case, recursion never stops → `StackOverflowError`

```
static int bad(int n) { return n * bad(n - 1); // NO base case! Infinite loop }
```

■ Mistake 2: Base Case Never Reached

The recursive call must always move towards the base case.

```
static int bad(int n) { if (n == 0) return 1; return n * bad(n + 1); // n grows!  
Never reaches 0 }
```

■ Mistake 3: Wrong Return Type

Always return the result of the recursive call, don't ignore it.

```
static int bad(int n) { if (n == 0) return 1; bad(n - 1); // Forgot to return!  
Result is lost return 1; // Always returns 1 }
```

- Always identify the BASE CASE first before writing any code.
- Trace small inputs (n=0, n=1, n=2) by hand to verify.
- Trust the recursion — assume the method works for smaller inputs.
- Print statements help trace what's happening during debugging.
- If confused, draw the call stack on paper step by step.

■ Practice Questions

Try writing these yourself — recursion gets easier with practice!

Easy ■

1. Write a recursive method to compute `power(base, exponent)` — e.g., `power(2,3) = 8`
2. Write a recursive method to count down from `n` to 1 and print each number.
3. Write a recursive method to find the minimum element in an array.

Medium ■■

1. Write a recursive method to check if a string is a palindrome (e.g., 'racecar').
2. Write a recursive method to compute the sum of digits of a number (e.g., `123 → 6`).
3. Write a recursive method to print all elements of a linked list in reverse.

Hard ■■■

1. Implement the Tower of Hanoi problem recursively for `n` disks.
2. Write a recursive method to generate all permutations of a string.
3. Write a recursive binary search method on a sorted array.

■ **Hint for Power:** Base case: `power(base, 0) = 1` Recursive: `power(base, exp) = base × power(base, exp-1)`

■ **Key Takeaway:** Every recursive method needs (1) a **Base Case** to stop, and (2) a **Recursive Case** that moves toward the base case. Master these two rules and you've mastered recursion!